

Módulo 01 — Como LLMs funcionam

Tokens, geração autoregressiva, sampling, context window, custo e alucinação. Com embeddings, atenção e pré-treino vs. RLHF/RLVR como leitura de apoio.

1. Visão geral
2. Conceitos
3. Exercício
4. Projeto final — etapa M1
5. Para revisar

Visão geral

Objetivos de aprendizagem

Ao final, a pessoa consegue:

1. explicar por que o mesmo prompt pode dar respostas diferentes, usando *geração autoregressiva* e *sampling* (temperature, top-p);
2. estimar quantos tokens um texto ocupa e quanto custa uma chamada, sabendo que entrada e saída têm preços diferentes;
3. dizer o que acontece quando uma conversa passa da *context window* e escolher o que descartar;
4. explicar por que o modelo alucina: ele prevê o texto mais plausível, não consulta fatos;
5. implementar softmax com temperatura e top-p, e ver na prática que um gerador de "próxima palavra" inventa frases plausíveis.

Mensagem central

Um LLM é uma função que recebe tokens e devolve uma **distribuição de probabilidade** sobre o próximo token. Todo o resto (o texto, a variação, o custo, a alucinação) vem de **sortear dessa distribuição, um token por vez**.

Conceitos

Este texto aprofunda a aula e cobre o que ficou de fora dela: **embeddings**, **atenção** e **pré-treino vs. RLHF/RLVR**. Leia na ordem; cada seção se apoia na anterior.

1. Tokens: a unidade que o modelo enxerga

O modelo não lê letras nem palavras. Antes de tudo, um **tokenizador** quebra o texto em pedaços frequentes e troca cada pedaço por um número (id):

```
"Parcelamento sem juros" → [Parcel][amento][ sem][ juros] → [ids...]
```

A técnica mais comum é a **BPE** (*Byte Pair Encoding*, adaptada para modelos de linguagem por [Sennrich et al., 2015](#)). Ela começa com caracteres e vai fundindo os pares mais frequentes do corpus de treino. Palavras comuns viram um token só; palavras raras viram vários.

Analogia: é como um teclado com teclas para sílabas e palavras frequentes, além das letras. "de" tem tecla própria; "inconstitucionalissimamente" precisa de várias.

Consequências práticas:

- **Custo e limite são em tokens**, não em palavras ou caracteres.
- **O idioma importa.** O mesmo conteúdo pode ocupar mais tokens em um idioma do que em outro, dependendo do tokenizador. [Petrov et al. \(2023\)](#) mediram diferenças de até 15x entre idiomas em alguns casos. A diferença afeta custo, latência e quanto cabe no contexto.
- **Cada família de modelos tem seu tokenizador.** Contagem de tokens feita com o tokenizador de um modelo é só uma estimativa para outro.
- **Coisas "óbvias" para humanos ficam difíceis.** Contar letras de uma palavra ou inverter uma string é estranho para quem enxerga `[Parcel][amento]`, não `P-a-r-c-e-l...`.

Para ver os pedaços: `pnpm --filter @mentoria/ex01-llms demo:tokens`, ou o visualizador [Tiktokenizer](#).

2. Embeddings: tokens viram vetores de significado *(não coberto em aula)*

O id de um token é só um número de catálogo; o id 1234 não está "perto" do 1235 em significado. O primeiro passo dentro do modelo é trocar cada id por um **embedding**: um vetor com centenas ou milhares de números, aprendido no treino.

Analogia: é um endereço num mapa de significados. Palavras usadas em contextos parecidos acabam em endereços próximos. O exemplo clássico, de [Mikolov et al. \(2013\)](#), é que as direções no espaço capturam relações:

```
vetor("rei") - vetor("homem") + vetor("mulher") ≈ vetor("rainha")
```

Dois usos que vão aparecer na trilha:

1. **Dentro do LLM**, o embedding é o ponto de partida. As camadas seguintes vão ajustando esse vetor conforme o contexto: "banco" perto de "juros" acaba num lugar diferente de "banco" perto de "praça".
2. **Fora do LLM**, *modelos de embedding* (como o `nomic-embed-text`, disponível no Ollama) transformam textos inteiros em vetores para busca por similaridade. É a base de RAG.

3. Transformer e atenção: como o contexto entra na conta *(não coberto em aula)*

O **transformer** ([Vaswani et al., 2017](#)) é a arquitetura dos LLMs atuais. A peça central é a **atenção**.

Intuição, sem matemática: para cada token, a atenção pergunta "quais outros tokens deste contexto importam para entender a mim?" e mistura a informação deles no seu vetor. Em "O cliente pediu o cartão, mas **ele** foi recusado", a atenção ajuda "ele" a puxar informação de "cartão" (e não de "cliente"), porque "recusado" combina mais com cartão.

Analogia: uma reunião em que cada pessoa (token), antes de falar, olha para a sala e decide quanto ouvir de cada colega. Isso se repete por dezenas de camadas, e a cada rodada a compreensão fica mais refinada.

Por que isso importa para quem usa LLM:

- **Cada token "vê" o contexto inteiro.** Por isso a ordem e a clareza do prompt importam.
- **O custo cresce com o contexto.** Na forma clássica, a atenção compara todos os pares de tokens. Contextos maiores custam mais memória e tempo, e por isso existe limite de janela.
- **"Ver" não é "usar bem".** [Liu et al. \(2023\)](#) mostraram que o desempenho cai quando a informação relevante está no meio de contextos longos, comparado ao começo ou ao fim.

Para aprofundar visualmente: [The Illustrated Transformer](#) (Jay Alammar) e as lições sobre [GPT e atenção](#) do 3Blue1Brown.

4. Geração autoregressiva: um token por vez

Depois das camadas, o modelo produz um **logit** (uma pontuação) para *cada token do vocabulário*. Esses logits viram probabilidades, um token é escolhido, entra no fim do contexto, e o processo se repete:

```
contexto → [modelo] → probabilidades do próximo token → escolhe 1 → contexto + token → [modelo] → ...
```

Isso explica três coisas que você sente no dia a dia:

- **Streaming:** o texto aparece aos poucos porque é gerado aos poucos.
- **Latência:** o prompt de entrada é processado de uma vez, em paralelo. Já a saída sai token por token, então respostas longas demoram mais.
- **Não há "plano" explícito:** cada token é escolhido olhando o que já foi escrito. Uma escolha ruim no início puxa as seguintes.

No desafio extra do exercício você implementa um gerador de bigramas que faz o mesmo jogo, só que olhando apenas a palavra anterior.

5. Sampling: como o próximo token é escolhido

Softmax com temperatura

Logits viram probabilidades com a **softmax**: $p_i = \exp(z_i / T) / \sum_j \exp(z_j / T)$, onde T é a *temperature*.

Temperature	Efeito	Exemplo com logits [1, 2, 3]
$T \rightarrow 0$	tudo no favorito (greedy)	[0, 0, 1]
$T = 0.5$	mais concentrado	[0.016, 0.117, 0.867]
$T = 1$	distribuição "original"	[0.090, 0.245, 0.665]
$T = 100$	quase uniforme	[0.33, 0.33, 0.33]

Analogia: a temperatura é o quanto você confia no favorito. Baixa = "vou de favorito sempre"; alta = "dou chance para os azarões".

Top-p (nucleus sampling)

Proposto por [Holtzman et al. \(2019\)](#): em vez de considerar o vocabulário inteiro, mantém o **menor conjunto de tokens mais prováveis que soma pelo menos p** e sorteia só entre eles. Corta a "cauda longa" de tokens improváveis, que é onde aparece o texto sem sentido, sem matar a variedade.

O artigo também mostra o problema oposto: escolher sempre o mais provável tende a gerar texto "sem graça e estranhamente repetitivo".

Regras práticas

- **Extração, classificação, código:** temperatura baixa (0 a 0,3). Você quer consistência.
- **Brainstorm, texto criativo:** temperatura mais alta. Você quer variedade.
- Ajuste **um** dos dois (temperature *ou* top-p) por vez.
- `temperature 0` reduz a variação, mas **não garante** a mesma saída em todos os provedores.

6. Context window

A **context window** é o máximo de tokens que o modelo processa de uma vez: instruções + histórico + documentos + a própria resposta.

- **O modelo não tem memória entre chamadas.** Num chat, o histórico inteiro é reenviado a cada mensagem, então o custo de entrada cresce a cada turno.
- **Quando estoura**, alguém precisa decidir o que sai: o provedor recusa, a biblioteca corta, ou você corta. O exercício implementa a estratégia mais simples (manter as instruções e as mensagens mais recentes). Alternativas: resumir o histórico antigo, ou buscar só os trechos relevantes (RAG).
- **Mais contexto não é sempre melhor:** além do custo, há a queda de desempenho com informação no meio (seção 3).

7. Custo e latência por token

Provedores cobram por token, geralmente com **preço diferente para entrada e saída** (saída costuma ser mais cara), em dólares por milhão de tokens:

```
custo = tokens_entrada × preço_entrada / 1.000.000 + tokens_saida × preço_saida / 1.000.000
```

Os preços mudam com frequência. Consulte sempre a página oficial do provedor, nunca um número de memória (nem deste material). Alguns provedores também cobram menos por tokens de entrada reaproveitados de chamadas anteriores (*prompt caching*).

Para latência, pense em duas partes:

- **Tempo até o primeiro token:** depende muito do tamanho da entrada (processada em paralelo).
- **Velocidade de saída (tokens/s):** como a saída é sequencial, o tempo total cresce com o tamanho da resposta.

Consequência de design: para tarefas de alto volume (como classificar centenas de notícias), peça respostas **curtas e estruturadas**. É exatamente o gancho do Módulo 2.

8. Pré-treino, SFT, RLHF e RLVR (*não coberto em aula*)

Um LLM de chat passa por fases de treino:

1. **Pré-treino:** prever o próximo token em uma quantidade enorme de texto. O resultado é um "modelo base" que completa texto muito bem, mas não necessariamente *segue instruções*. Pergunte "qual a capital da França?" e ele pode continuar com outras perguntas de prova, porque é assim que esse texto costuma continuar.
2. **SFT (supervised fine-tuning):** treinar com exemplos de instrução → resposta boa, escritos ou revisados por pessoas.
3. **RLHF (reinforcement learning from human feedback):** pessoas comparam respostas; um modelo de recompensa aprende essas preferências; o LLM é otimizado para agradar esse modelo. É a receita do InstructGPT ([Ouyang et al., 2022](#)).
4. **RLVR (reinforcement learning with verifiable rewards):** em vez de preferência humana, a recompensa vem de uma **verificação automática**: a resposta da conta bate? o código passa nos testes? O nome foi cunhado no Tulu 3 ([Lambert et al., 2024](#)), e a ideia de recompensas baseadas em regras foi central para modelos de raciocínio como o DeepSeek-R1 ([DeepSeek-AI, 2025](#)).

Analogia: pré-treino é ler a biblioteca inteira; SFT é estudar com exemplos resolvidos; RLHF é ter um professor dizendo qual de duas redações ficou melhor; RLVR é fazer uma lista de exercícios com gabarito.

O Módulo 2 apresenta uma quinta variação, o RLCD, usado pela TypeSafe AI para treinar modelos que devolvem decisões com probabilidades calibradas.

9. Alucinação: por que acontece

Alucinação é quando o modelo gera uma afirmação plausível e falsa com a mesma fluência de uma verdadeira.

Pelo mecanismo das seções anteriores, isso é esperado: o modelo gera **o texto mais plausível dado o contexto**, e plausível não é o mesmo que verdadeiro. Se a pergunta cita um artigo que não existe, a continuação mais plausível de "Resuma o artigo X" é... um resumo.

Kalai et al. (2025) dão uma explicação estatística em duas partes:

- **No pré-treino**, se o modelo não consegue distinguir afirmações incorretas de fatos, erros surgem por pressão estatística natural. Os autores tratam isso como erro de classificação binária.
- **Depois do treino**, a alucinação persiste porque a maioria das avaliações **premia chutar** em vez de admitir incerteza: como numa prova de múltipla escolha sem desconto por erro, "chutar" melhora a nota. A proposta deles é mudar a forma como os benchmarks pontuam.

O que isso significa para quem constrói sistemas:

- **Não dá para confiar só no modelo** para fatos. Forneça a fonte no contexto e exija que a resposta cite a fonte.
- **Valide a saída** com schema, regras e checagem de URL (Módulo 3).
- **Use a incerteza a seu favor**: um sinal de confiança permite mandar casos duvidosos para revisão humana (Módulos 2 e 7).

Referências

- Sennrich, Haddow, Birch (2015). *Neural Machine Translation of Rare Words with Subword Units*. <https://arxiv.org/abs/1508.07909>
- Petrov et al. (2023). *Language Model Tokenizers Introduce Unfairness Between Languages*. <https://arxiv.org/abs/2305.15425>
- Karpathy. *Let's build the GPT Tokenizer*. <https://www.youtube.com/watch?v=zduSFxRajkE>
- Mikolov et al. (2013). *Efficient Estimation of Word Representations in Vector Space*. <https://arxiv.org/abs/1301.3781>
- Vaswani et al. (2017). *Attention Is All You Need*. <https://arxiv.org/abs/1706.03762>
- Alammr. *The Illustrated Transformer*. <https://jalammar.github.io/illustrated-transformer/>
- 3Blue1Brown. *Transformers (how LLMs work) e Attention in transformers*. <https://www.3blue1brown.com/lessons/gpt> · <https://www.3blue1brown.com/lessons/attention>
- Holtzman et al. (2019). *The Curious Case of Neural Text Degeneration*. <https://arxiv.org/abs/1904.09751>
- Liu et al. (2023). *Lost in the Middle: How Language Models Use Long Contexts*. <https://arxiv.org/abs/2307.03172>

- Ouyang et al. (2022). *Training language models to follow instructions with human feedback*. <https://arxiv.org/abs/2203.02155>
- Lambert et al. (2024). *Tulu 3: Pushing Frontiers in Open Language Model Post-Training*. <https://arxiv.org/abs/2411.15124>
- DeepSeek-AI (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. <https://arxiv.org/abs/2501.12948>
- Kalai et al. (2025). *Why Language Models Hallucinate*. <https://arxiv.org/abs/2509.04664>

Exercício

Tarefa assíncrona. Não precisa de modelo rodando: tudo aqui é offline e determinístico.

Você vai implementar, em TypeScript puro, as peças que a aula mostrou em funcionamento: contar tokens e estimar custo, transformar logits em probabilidades com temperatura, cortar a cauda com top-p, sortear o próximo token e encaixar uma conversa na context window.

Como rodar

Da raiz do repositório:

```
pnpm --filter @mentoria/ex01-llms test          # versão base
pnpm --filter @mentoria/ex01-llms test:desafio  # desafio extra
```

Um teste só, enquanto você trabalha:

```
pnpm --filter @mentoria/ex01-llms exec vitest run test/sampling.test.ts -t "softmax"
```

Os testes **falham** até você implementar. É esperado.

A solução de referência está em [solucao/](#). Tente antes de abrir. Para rodar os mesmos testes contra ela: `pnpm --filter @mentoria/ex01-llms test:solucao`.

Versão base

Implemente as funções marcadas com `TODO`. O comentário de cada função descreve o comportamento exato que os testes verificam.

Arquivo	Função	O que faz
<code>src/tokens.ts</code>	<code>contarTokens</code>	conta tokens com o tokenizador fornecido (<code>src/tokenizador.ts</code>)
	<code>razaoDeTokens</code>	quantas vezes mais tokens o texto A ocupa em relação ao B
	<code>custoPorChamada</code>	custo em dólares, com preços diferentes para entrada e saída

Arquivo	Função	O que faz
src/sampling.ts	softmax	logits → probabilidades, com temperatura (0 = greedy)
	filtrarTopP	nucleus sampling: zera a cauda e renormaliza
	amostrar	softmax → top-p → sorteio pelo acumulado
src/contexto.ts	ajustarAoContexto	mantém o <code>system</code> e as mensagens mais recentes que cabem no limite

Sugestão de ordem: `tokens.ts` → `softmax` → `filtrarTopP` → `amostrar` → `contexto.ts`.

Dicas:

- Na `softmax`, subtraia o maior logit antes de `Math.exp`. O resultado é o mesmo e não estoura.
- No `filtrarTopP`, ordene uma *cópia* dos índices por probabilidade; não reordene o array em si.
- O `amostrar` recebe a função `aleatorio` por parâmetro para os testes serem reproduzíveis. Nunca chame `Math.random` diretamente dentro dele.

Feito = `pnpm --filter @mentoria/ex01-llms test` com todos os testes passando.

Desafio extra

Em `src/desafio/bigrama.ts`, construa um mini modelo de linguagem de **bigramas**: ele aprende, num corpus de frases, quantas vezes cada palavra vem depois de outra, e gera frases novas **palavra a palavra** usando o seu `amostrar`.

É o mesmo jogo de um LLM (prever o próximo token), só que olhando apenas a palavra anterior. O último teste mostra o ponto da aula: **toda transição que ele gera existe no corpus, e mesmo assim ele inventa frases que nunca viu**. Por exemplo, ele pode juntar "o brasil tem capital em" com "paris". É uma alucinação em miniatura: localmente plausível, globalmente falsa.

Feito = `pnpm --filter @mentoria/ex01-llms test:desafio` com todos os testes passando.

Para ir além (sem teste):

- Gere 20 frases com `temperature` 0, 1 e 2 e compare.
- Troque o corpus por algumas frases suas. O que acontece com corpus pequeno vs. grande?

Perguntas para pensar

1. Na sua `softmax`, o que acontece com a probabilidade do favorito quando a temperatura tende a 0? E a infinito?

2. Por que `filtrarTopP` precisa renormalizar depois de zerar a cauda?
3. Em `ajustarAoContexto`, por que não pular uma mensagem grande para encaixar uma mais antiga e menor? Que problema de coerência isso causaria?
4. O seu modelo de bigramas "sabe" que Paris não é a capital do Brasil? O que um LLM tem a mais, e o que continua igual?

Projeto final — etapa M1

Etapa M1 — classificação em texto livre

Objetivo: um prompt que diz se a notícia é relevante para a newsletter e em que categoria ela entra (modelos, ferramentas, pesquisa, regulacao, mercado).

Implemente em `src/m01/classificar-livre.ts`:

1. `montarPromptClassificacao(noticia) → { system, prompt }`
 - `system`: papel do modelo, critério de relevância (veja `dados/README.md`) e **todas** as categorias.
 - `prompt`: título, resumo e fonte da notícia.
2. `classificarLivre(noticia, { temperature, modelo }) → texto da resposta`, chamando `gerarTexto` de `@mentoria/llm`.

Feito =

1. `pnpm --filter @mentoria/curador exec vitest run test/m01.test.ts` verde (offline, com mock);
2. o experimento roda com o modelo de verdade e grava `projeto-final/saidas/m01-temperaturas.md`:

```
pnpm --filter @mentoria/curador m01:temperaturas
```

Abra o arquivo gerado e responda (vamos discutir na Aula 2):

- As respostas mantêm o **mesmo formato** entre execuções? E entre temperaturas?
- Quantas acertaram o rótulo humano?
- Se outro programa precisasse ler essa resposta para decidir se publica a notícia, o que poderia dar errado?

Desafio extra: sem mudar a assinatura de `classificarLivre`, escreva um `interpretarResposta(texto)` que extraia `relevante` e `categoria` da resposta livre com regex ou parsing. Rode no arquivo de saída e conte quantas respostas você conseguiu interpretar. Guarde o número: na Aula 2 vamos compará-lo com uma saída estruturada.

Para revisar

Perguntas para discussão (15 min)

1. Se `temperature 0` dá sempre a mesma resposta, por que não usar 0 sempre? Em que tarefa do seu dia a dia você quer variação?
2. O projeto final vai classificar notícias. Que temperatura você usaria? Por quê?
3. Um chatbot de atendimento "esqueceu" o que o cliente disse no começo da conversa. Quais são as explicações possíveis, à luz da context window?
4. "O modelo mentiu." Essa frase faz sentido? Qual seria uma descrição mais precisa do que aconteceu?
5. Se saída custa mais que entrada e é gerada um token por vez, o que muda no design de um prompt para uma tarefa de alto volume?

Armadilhas comuns

- **Achar que `temperature 0` é determinístico em qualquer provedor.** Em muitos é *quase*: detalhes de hardware, batching e versão do modelo podem mudar o resultado. Não use isso como garantia em teste.
- **Contar palavras em vez de tokens** para estimar custo ou limite de contexto.
- **Usar o tokenizador errado:** cada família de modelos tem o seu, e a contagem muda.
- **Esquecer que o histórico da conversa é reenviado a cada chamada:** o custo de entrada cresce a cada turno.
- **Tratar alucinação como bug raro.** É consequência direta do mecanismo; a mitigação vem de fora do modelo (fontes, validação, evals), assunto dos Módulos 2 e 3.
- **Mexer em `temperature` e `top-p` ao mesmo tempo** sem saber qual efeito veio de qual.